CH 12

REDUCING WORK AND ELIMINATING EXCISE

Digital products too often contain interactions that are top-heavy, requiring unnecessary work for users. Interacting with an interface always involves some work on the part of the user. The goal of designers (or at least one of the more important ones) is to minimize that work, while at the same time enabling users to achieve their goals. If designers and developers don't pay careful attention to the human actions required to operate their technology, the result can be a taxing experience for the users. They will struggle to relate their mental models of the activities they want to perform to the product interface that has been engineered.

Users perform four types of work when interacting with digital products:

- **Cognitive work**—Comprehending product behaviors, as well as text and organizational structures
- **Memory work**—Recalling product behaviors, commands, passwords, names and locations of data objects and controls, and other relationships between objects
- **Visual work**—Figuring out where the eye should start on the screen, finding one object among many, decoding layouts, and differentiating among visually coded interface elements (such as list items with different colors)
- **Physical work**—Keystrokes, mouse movements, gestures (click, drag, double-click), switching between input modes, and number of clicks required to navigate

When implementation-model thinking is applied to digital products, these four types of work are seldom minimized for users—quite the opposite, in fact. The result is software that, in effect, charges its users a tax, or **excise**, of cognitive and physical effort every time it is used.

In the physical world, mandatory tasks that don't immediately satisfy our goals are sometimes unavoidable. For example, when we get up late on a workday and need to get to the office quickly, we must open the garage door, get in the car, start the motor, back out, and close the garage door before we even begin the forward motion that will take us to our destination. These actions support the physicality of the automobile rather than getting us to the destination faster.

If we had *Star Trek* transporters instead, we'd dial up our destination and teleport there instantaneously—no garages, no motors, no traffic lights. Digital products, much like our fictional transporter, don't necessarily need to have the same kind of roadblocks that stand in the way of our goals in the physical world. But implementation-model design often makes it seem that way to users.

Goal-Directed Tasks versus Excise Tasks

Any large task, such as driving to the office, involves many smaller tasks. Some of these tasks work directly toward achieving the goal; these are tasks like steering down the road toward your office. *Excise tasks*, on the other hand, don't contribute directly to reaching the goal, but instead represent extra work that satisfies either the needs of our tools or those of outside agents as we try to achieve our objectives.

In this example, the excise tasks are pretty clear. Opening the garage door is something we do for the car, not for us, and it doesn't move us toward our destination the way the accelerator pedal and steering wheel do. Stopping at red lights is something imposed on us by our society that, again, doesn't help us achieve our true goal. (In this case, it does help us achieve a related goal of arriving *safely* at our office.) A tune-up helps keep the car running well, but it doesn't get us anywhere quickly while we're doing it.

Software, too, has a pretty clear dividing line between *goal-directed tasks* and excise tasks. Like automobiles, some software excise tasks are trivial, and performing them is no great hardship. On the other hand, other software excise tasks are as obnoxious as fixing a flat tire. Installation leaps to mind here, as do such excise tasks as configuring networks and backing up files.

Types of Excise

The problem with excise tasks is that the effort we expend doing them doesn't go directly toward accomplishing our goals. Where we can eliminate excise tasks, we make people more effective and productive, ultimately creating better usability and a better user experience.

DESIGN PRINCIPLE

Eliminate excise wherever possible.

The existence of excise in user interfaces is a primary cause of user dissatisfaction with software-enabled products. It behooves every designer and product manager to be on the lookout for interaction excise in all its forms and to take the time and energy to see that it is eliminated from their products.

Navigational excise

Navigation through the functions or features of a digital product is largely excise. Except in the case of games where the *goal* is to navigate successfully through a maze of obstacles, the work that users are forced to do to get around in software and on websites is seldom aligned with their needs, goals, and desires. (However, well-designed navigation can be an effective way to instruct users about what is available to them, which is better aligned with their goals.)

Unnecessary or difficult navigation is a major frustration to users. In fact, in our opinion, poorly designed navigation presents one of the largest and most common problems in the usability of interactive products—mobile, desktop, web, or otherwise. It is also the place where the developer's implementation model typically is made most apparent to users.

Navigation through software occurs at multiple levels:

- Across multiple windows, views, or pages
- Across multiple panes or frames within a window, view, or page
- Across tools, commands, or menus
- Within information displayed in a pane or frame (such as scrolling, panning, zooming, following links)

We find it useful to think in terms of a broad definition of navigation: *any action that takes the user to a new part of the interface or that requires him or her to locate objects, tools, or data elsewhere in the system*. When we start thinking about such actions as navigation, it becomes clear that they are excise and therefore should be minimized or, if possible, eliminated. The following sections discuss each of these types of navigation in more detail.

Navigation across multiple screens, views, or pages

Moving across multiple application views or pages is perhaps the most disorienting kind of navigation for users. It involves a gross shifting of attention that disrupts the user's flow and forces him into a new context. The act of navigating to another window also often means that the contents of the original window are partly or completely obscured. On the desktop, it means that the user needs to worry about window management, an excise task that further disrupts his flow. If users must constantly shuttle between windows to achieve their goals, their disorientation and frustration levels will rise, they will become distracted from the task at hand, and their effectiveness and productivity will drop.

If the number of windows is large enough, the user will become sufficiently disoriented that he may experience *navigational trauma*: He gets lost in the interface. Sovereign posture applications (discussed in Chapter 9) can avoid this problem by placing all main interactions in a single primary view, which may contain multiple independent panes.

Navigation between panes

Windows or views can contain multiple panes—adjacent to each other and separated by splitters (see Chapter 20) or stacked on top of each other and denoted by tabs. Adjacent panes can solve many navigation problems, because they provide useful supporting functions, links, or data on the screen in close reach of the primary work or display area. This reduces navigation to almost nil. If objects can be dragged between panes, those panes should be adjacent.

Problems arise when adjacent supporting panes become too numerous or are not placed on the screen in a way that matches users' work flows. Too many adjacent panes results in visual clutter and confusion: Users do not know where to go to find what they need. Also, crowding forces scrolling—another navigational hit. Navigation within the single screen thus becomes a problem. Some web portals, trying to be everything to everyone, have such navigational problems.

In some cases, depending on user work flows, tabbed panes can be appropriate. Tabbed panes bring with them a level of navigational excise and potential for user disorientation

because they obscure what was on the screen before the user navigated to them. However, this idiom is appropriate for the main work area when multiple documents or independent views of a document are required (such as in Microsoft Excel; see Figure 12-1).

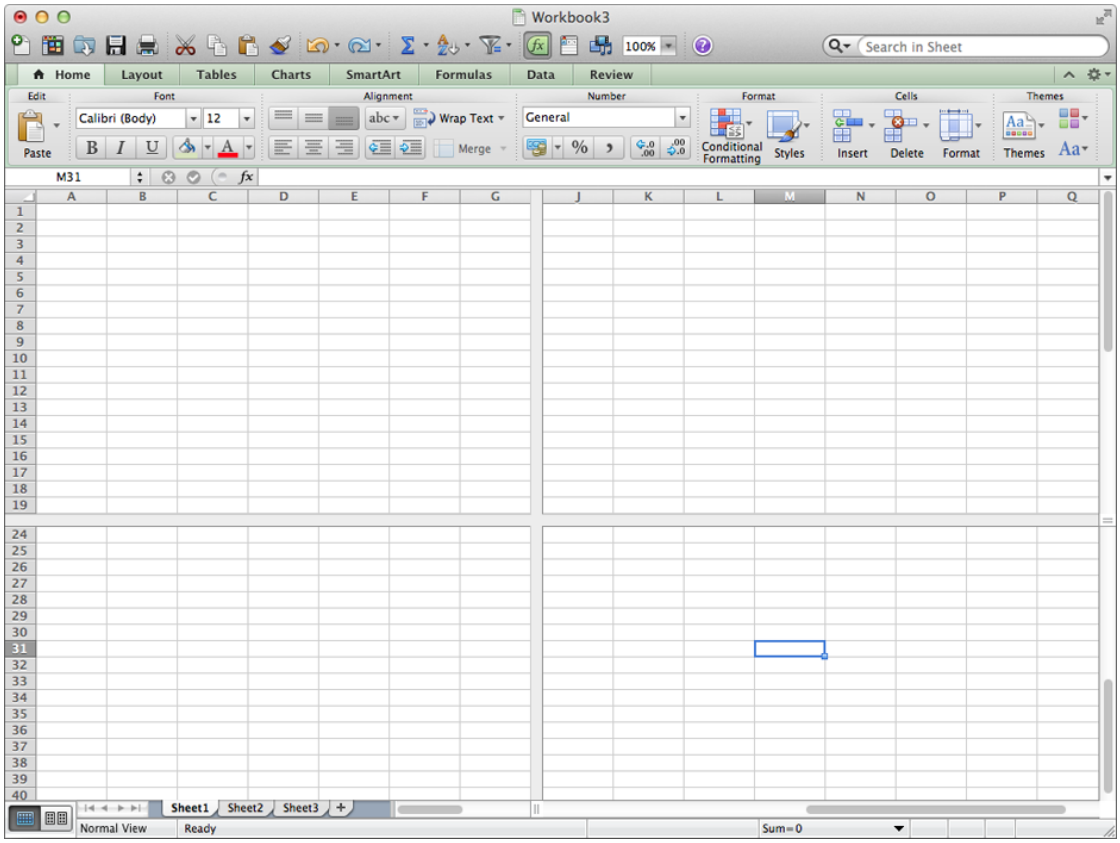


Figure 12-1: Microsoft Excel makes use of tabbed panes (visible in the lower left) to let users navigate between related worksheets. Excel also makes use of splitters to provide adjacent panes for viewing multiple, distant parts of a single spreadsheet without constant scrolling. Both these idioms help reduce navigational excise for Excel users.

Some developers use tabs to break complex product capabilities into smaller chunks. They reason that using these capabilities will somehow become easier if the functionality is cut into bite-sized pieces. Actually, putting parts of a single facility onto separate panes increases excise and decreases users' understanding and orientation.

The use of tabbed screen areas is a space-saving mechanism and is sometimes necessary to fit all the required information and functions into a limited space. (Settings dialogs are a classic example. We don't think anyone is interested in seeing all the settings for a sophisticated application laid bare in a single view.) In most cases, though, the use of tabs creates significant navigational excise. It is rarely possible to accurately describe the contents of a tab with a succinct label (though in a pinch, *rich visual modeless feedback*

on tabs can help—see Chapter 15). Therefore, users must click through each tab to find the tool or piece of information they are looking for.

Tabbed panes can be appropriate when there are multiple supporting panes for a primary work area that are not used at the same time. The support panes can be stacked, and the user can choose the pane suitable for his current tasks, which is only a click away. A classic example involves the color mixer and swatches area in Adobe Illustrator, as shown in Figure 12-2. These two tools are mutually exclusive ways of selecting a drawing color, and users typically know which is appropriate for a given task.

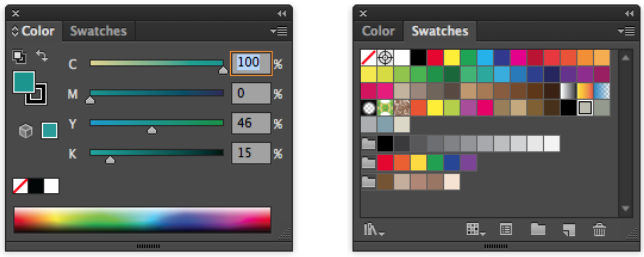


Figure 12-2: Tabbed palettes in Adobe Illustrator allow users to switch between the mixer and swatches, which provide alternative mechanisms for picking a color.

Navigation between tools and menus

Another important and overlooked form of navigation results from a user's need to use different tools, palettes, and functions. Spatial organization of these within a pane or window is critical to minimizing extraneous mouse movements that, at best, could result in user annoyance and fatigue and, at worst, could result in repetitive stress injury. Tools that are used frequently and in conjunction with each other should be grouped spatially and also should be immediately available. Menus require more navigational effort on the part of users because their contents are not visible prior to clicking. Frequently used functions should be provided in toolbars, palettes, or the equivalent. Menu use should be reserved for infrequently accessed commands. (We discuss organizing controls again later in this chapter, and we discuss toolbars in depth in Chapter 18.)

Adobe Photoshop exhibits some undesirable behaviors in how it forces users to navigate between palette controls. For example, the Paint Bucket tool and the Gradient tool each occupy the same location on the tool palette. You must select between them by clicking and holding the visible control, which opens a menu, as shown in Figure 12-3. However, both are fill tools, and if both are used frequently, it would be better to place each of them on the palette next to each other to avoid that frequent, flow-disrupting tool navigation.

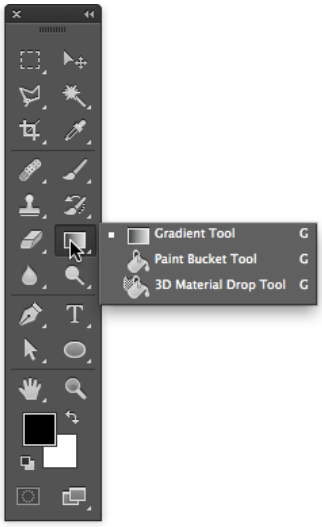


Figure 12-3: In Adobe Photoshop, the Paint Bucket tool is hidden in a combo icon button (see Chapter 21) on its tool palette. Even though users make frequent use of both the Gradient tool and the Paint Bucket tool, they are forced to access this menu anytime they need to switch between these tools.

Navigation of information

Information, or the content of panes or windows, can be navigated using several methods: scrolling (panning), linking (jumping), and zooming. The first two methods are common: Scrolling is ubiquitous in most software, and linking is ubiquitous on the web (although increasingly, linking idioms are being adopted in non-web applications). Zooming is used primarily to visualize 3D and detailed 2D data.

Scrolling is often a necessity, but the need for it should be minimized when possible. Often there is a trade-off between paging and scrolling information: You should understand your users' mental models and work flows to determine what is best for them.

In 2D visualization and drawing applications, vertical and horizontal scrolling are common. These kinds of interfaces benefit from a thumbnail map to ease navigation. We'll discuss this technique as well as other visual signposts later in this chapter.

Linking is the critical navigational paradigm of the web. Because it is a visually dislocating activity, extra care must be taken to provide visual and textual cues that help orient users.

Zooming and *panning* are navigational tools for exploring 2D and 3D information. These methods are appropriate when creating 2D or 3D drawings and models or for exploring

representations of real-world 2D and 3D environments (architectural walkthroughs, or topographic maps, for example). They can fall short when they are used to examine arbitrary or abstract data presented in more than two dimensions. Some information visualization tools use zoom to mean "Display more attribute details about objects"—a logical rather than spatial zoom. As the view of the object enlarges, attributes (often textual) appear superimposed over its graphical representation. This technique works great when the attributes in question are tightly associated with spatial data, such as that employed in Google Maps (see Figure 12-4). But for abstract data spaces, this kind of interaction is almost always better served through an adjacent supporting pane that displays the properties of selected objects in a more standard, readable form.

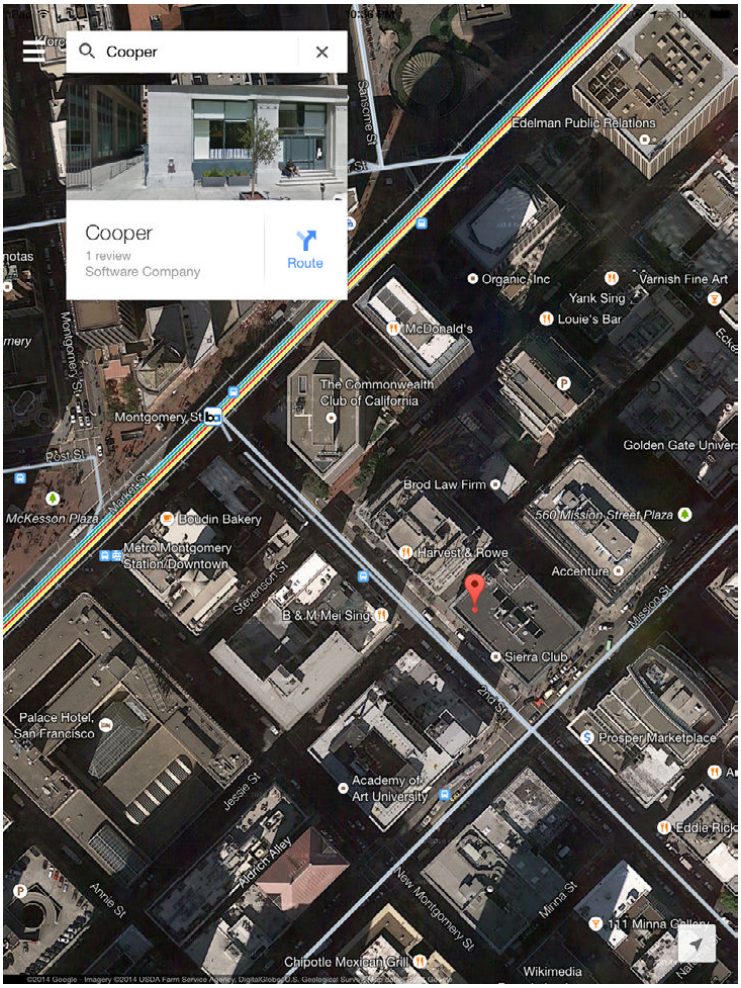


Figure 12-4: The Google Maps app makes excellent use of a combination of spatial and logical zoom. As the user physically zooms in by spreading his fingers apart on the map, location details such as transit lines, traffic congestion, street names, and places of business also come into view. Zoom usually works best when applied to concrete rather than abstract data spaces, such as maps.

Panning and zooming, especially when paired, create navigational difficulties for users. Although this situation is improving due to the prevalence of online maps and easy to grasp gestural interfaces, it is still possible for people to get lost using virtual spaces. Humans are not used to moving in unconstrained 3D environments, and they have difficulty perceiving 3D properly when it is projected on a 2D screen. (See Chapter 18 for more on 3D manipulation.)

Skeuomorphic excise

We are experiencing an incredible transformation from the age of industrial, mechanical artifacts to an age of digital, information objects. It is only natural for us, then, to try to draw on the models and forms of an earlier era that we are comfortable with and use them in this new, less certain one. As the history of the industrial revolution shows, the fruits of new technology can often only be expressed at first with the language of an earlier technology. For example, we called railroad engines iron horses and automobiles horseless carriages. Unfortunately, this imagery and language color our thinking more than we might admit.

Naturally, we tend to use old-style mechanical representations in our new digital environments, a practice called *skeuomorphism*. Sometimes this appropriation of the old is valid because the function is identical, even if the underlying technology is different. For example, when we translate the process of typing with a typewriter into doing word processing on a computer, we are using a mechanical representation of a common task. Typewriters used little metal tabs to rapidly move the carriage several spaces until it came to rest on a particular column. The process, as a natural outgrowth of the technology, was called tabbing or setting tabs. Word processors also have tabs because their function is the same; whether you are working on paper rolled around a platen or on images on a video screen, you need to rapidly slew to a particular margin offset.

More often, however, mechanical representations shouldn't be translated verbatim into the digital world. We encounter problems when we bring our familiar mechanical artifacts into software. These representations result in excise and unnecessarily limit interactions that could be far more efficient than those allowed for by the old models.

Mechanical procedures are usually easier to perform by hand than they are with digital products. Consider a simple contact list. If it is faithfully rendered onscreen like a little bound book, it will be much more complex, inconvenient, and difficult to use than the physical address book. The physical address book, for example, stores names in alphabetical order by last name. But what if you want to find someone by her first name? The mechanical artifact doesn't help you: You have to scan the pages manually. The faithfully replicated digital version wouldn't search by first name either. The difference is that,

on the computer screen, you lose many subtle visual and tangible cues offered by the paper-based book (bent page corners, penciled-in notes). Meanwhile, scrollbars, swipe-to-delete, and navigational drilldowns are harder to use, harder to visualize, and harder to understand than simply flipping pages.

Designers paint themselves into a corner when they rely on slavishly skeuomorphic metaphors. Visual metaphors such as desktops with telephones, copy machines, staplers, and fax machines—or file cabinets with folders in drawers—may make it easy to understand the relationships between interface elements and behaviors. But after users learn these fundamentals, managing the metaphor becomes an exercise in excise. (For more discussion on the limitations of visual metaphors, see Chapter 13.)

Screen real estate consumed by skeuomorphic representations is also excessive, particularly in sovereign posture applications, where maximizing screen space for content rather than UI chrome is of primary importance. The little telephone that so charmingly told us how to dial on that first day long ago is now just a barrier to quick communication.

It's all too easy to fall into the trap of skeuomorphic excise in the name of user friendliness. Apple's iOS veered uncomfortably in the direction of skeuomorphism for versions 4, 5, and 6, but it seems to have finally snapped out of it in iOS 7, as shown in Figure 12-5.

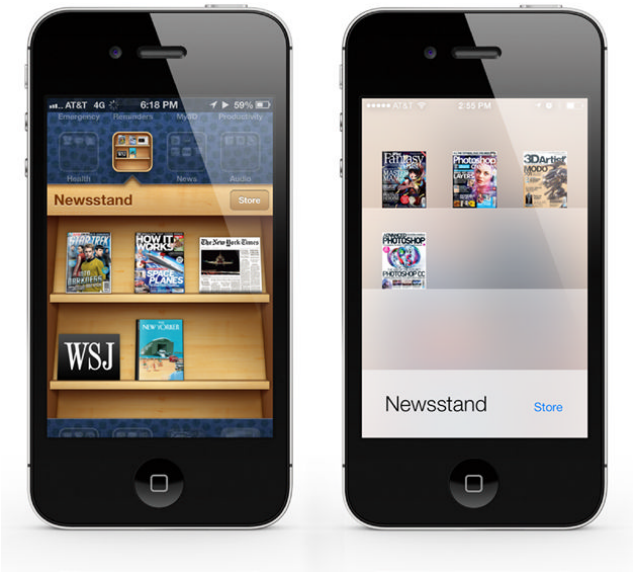


Figure 12-5: In iOS 6 (left), Apple indulged in some excesses of skeuomorphism that seem to have been purged in iOS 7 (right).

Modal excise

The previous chapter introduced the concept of *flow*, whereby a person enters a highly productive mental state by working in harmony with her tools. Flow is a natural state, and people enter it without much prodding. It takes some effort to break into flow after someone has achieved it. Interruptions like a ringing telephone will do it, as will a modal error message or confirmation dialog. Some interruptions are unavoidable, but interrupting the user's flow for no good reason is *stopping the proceedings with idiocy* and is one of the most disruptive forms of excise.

DESIGN PRINCIPLE

Don't stop the proceedings with idiocy.

Poorly designed software makes assertions that no self-respecting individual would ever make. It states unequivocally, for example, that a file doesn't exist merely because the software is too stupid to look for the file in the right place, and then it implicitly blames *you* for losing it! An application cheerfully executes an impossible query that hangs up your system until you decide to reboot. Users view such software behavior as idiocy, and with just cause.

Errors, notifiers, and confirmation messages

There are probably no more prevalent excise elements than error message and confirmation message dialogs. These are so ubiquitous that eradicating them takes a lot of work. In Chapter 15, we discuss these issues at length, but for now, suffice it to say that they are high in excise and should be eliminated from your applications whenever possible.

The typical modal error message is unnecessary. It either tells the user something he doesn't care about or demands that he fix a situation that the application can and should usually fix just as well. Figure 12-6 shows an error message box displayed by Adobe Illustrator 6 when the user tries to save a document. We're not exactly sure what it's trying to tell us, but it sounds dire.

The message stops an already annoying and time-consuming procedure, making it take even longer. A user cannot fetch a cup of coffee after telling the application to save his artwork, because he might return only to see the function incomplete and the application mindlessly holding up the process. We discuss how to eliminate these sorts of error messages in Chapter 21.

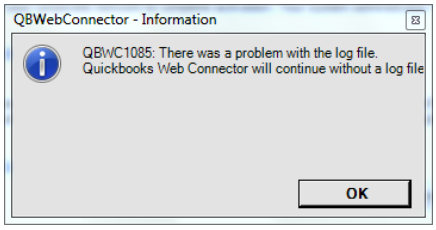


Figure 12-6: This ugly, useless error message box stops the proceedings with idiocy. You can't verify or identify what it tells you, and it gives you no options for responding other than to admit your own culpability by clicking OK. This message comes up only when the application is saving—when you have entrusted it to do something simple and straightforward. The application can't even save a file without help, and it won't tell you what help it needs!

Figure 12-7 shows another frustrating example, this time from Microsoft Outlook.

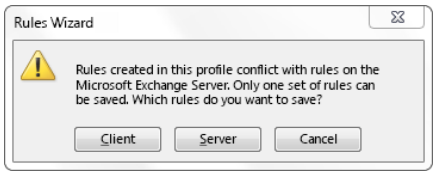


Figure 12-7: Here is a horrible confirmation box that stops the proceedings with idiocy. If the application is smart enough to detect the difference, why can't it correct the problem itself? The options the dialog offers are scary. It is telling you that you can explode one of two boxes: One contains garbage, and the other contains the family dog—but the application won't say which is which. And if you click Cancel, what does that mean? Will it still explode your dog?

This dialog is asking you to make an irreversible and potentially costly decision based on no information whatsoever! If the dialog occurs just after you changed some rules, doesn't it stand to reason that you want to keep them? And if you don't, wouldn't you like a bit more information, such as exactly what rules are in conflict and which of them are the more recently created? You also don't have a clear idea what happens when you click Cancel. Are you canceling the dialog and leaving the rules mismatched? Are you discarding recent changes that led to the mismatch? The kind of fear and uncertainty that this poorly designed interaction arouses in users is completely unnecessary. We discuss how to improve this kind of situation in Chapter 21.

Making users ask permission

Back in the days of command lines and character-based menus, interfaces indirectly offered services to users. If you wanted to change an item, such as your address, first you had to ask the application for permission to do so. The application would then display a screen where you could change your address. Asking permission is pure excise, and unfortunately things haven't changed much. If you want to change one of your saved addresses on Amazon.com, you have to click a button and go to a different page. If you want to change a displayed value, you should be able to change it right there. You shouldn't have to ask permission or go to a different room.

DESIGN PRINCIPLE *Don't make users ask permission.*

As in the preceding example, many applications have one place where the values (such as filenames, numeric values, and selected options) are displayed for output and another place where user input to them is accepted. This follows the implementation model, which treats input and output as different processes. A user's mental model, however, doesn't recognize a difference. He thinks, "There is the number. I'll just click it and enter a new value." If the application can't accommodate this impulse, it is needlessly inserting excise into the interface. If the user can modify options, he should be able to do so right where the application displays them.

DESIGN PRINCIPLE *Allow input wherever you have output.*

The opposite of asking permission can be useful in certain circumstances. Rather than asking the application to launch a dialog, the user tells a dialog to go away and not return. In this way, the user can make an unhelpful dialog stop badgering him, even though the application mistakenly thinks it is helping. Microsoft Windows now makes heavy use of this idiom. (If a beginner inadvertently dismisses a dialog and can't figure out how to get it back, he may benefit from another easy-to-identify safety-net idiom in a prominent place—a Help menu item saying "Bring back all dismissed dialogs," for example.)

Stylistic excise

Users must perform visual work to decode onscreen information, such as finding a single item in a list, figuring out where to begin reading on a screen, or determining which elements on a screen are clickable and which are merely decoration.

A significant source of visual work is the use of overly stylized graphics and interface elements (see Figure 12-8). Visual style can certainly create mood and reinforce brand, but it shouldn't do so at the expense of utility and usability by forcing users to decode visual elements to understand which represent controls and critical information and which are merely ornamental. The use of visual style, at least in apps geared toward productivity rather than entertainment, should support the clear communication of information and interface behavior.

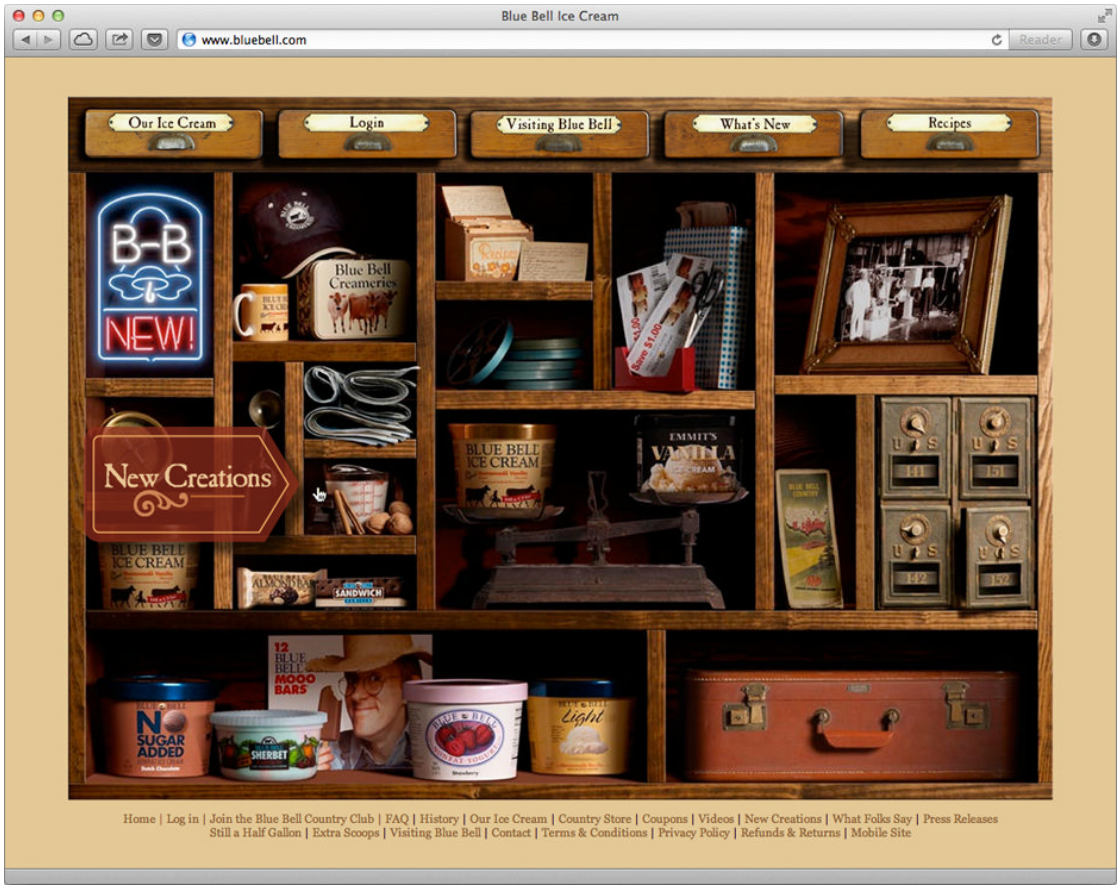


Figure 12-8: The home page of Blue Bell Creameries provides a good example of visual excise. Text is highly stylized and doesn't follow a layout grid. It's difficult for users to differentiate between décor and navigational elements. This requires users to do visual work to interact with the site. This isn't always a bad thing—just the right amount of the right kind of work can be a source of entertainment (as with games and puzzles).

For more discussion on striking the right balance to create effective visual interface designs, see Chapter 17.

Excise Is Contextual

One man's (or persona's) goal-directed task may be another's excise task in a different context. In general, a function or action is excise if it is forced on the user rather than made available at his discretion. An example of this kind of function is window management. The only way to determine whether a function or behavior such as this is excise is by comparing it to personas' goals. If a significant persona needs to see two applications at a time on the screen to compare or transfer information, the ability to configure the main windows of the applications so that they share the screen space is not excise. If your personas don't have this specific goal or need, the work required to configure the main window of either application *is* excise.

Excise may also vary by software posture (see Chapter 9). Users of transient posture applications often require some instruction to use the product effectively. Allocating screen real estate to this effort typically does not contribute to excise in the same way as it does in sovereign posture applications. Transient posture applications aren't used frequently, so their users need more assistance understanding what the application does and remembering how to control it. For sovereign posture applications, however, the slightest excise becomes agonizing over time.

However, some types of actions are almost *always* excise and should be eliminated under all circumstances. These include most hardware-management tasks that the software could handle itself (if a few more design and engineering cycles were spent on it). Any demands for such information should be struck from user interfaces and replaced with more silently intelligent application behavior behind the scenes.

Eliminating Excise

Navigational excise is easily the most prevalent type of excise found in digital products and thus is one of the best places to start eliminating it. There are many ways to begin improving (eliminating, reducing, or speeding up) navigation in your applications, websites, and devices. Here are the most effective:

- Reduce the number of places to go.
- Provide signposts.
- Provide overviews.
- Properly map controls to functions.
- Avoid hierarchies.
- Don't replicate mechanical models.

We'll discuss these in detail next.

Reduce the number of places to go

The most effective method of improving navigation sounds quite obvious: Reduce the number of places to which one must navigate. These "places" include modes, forms, dialogs, pages, windows, and screens. If the number of modes, pages, or screens is kept to a minimum, people's ability to stay oriented increases dramatically. In terms of the four types of navigation presented earlier, this directive means you should do the following:

- Keep the number of windows and views to a minimum. One full-screen window with two or three views is best for many users. Keep dialogs, especially modeless dialogs, to a minimum. Applications, websites, or mobile apps with dozens of distinct types of pages, screens, or forms are difficult to navigate.
- Limit the number of adjacent panes in your interface to the minimum number needed for users to achieve their goals. In sovereign posture applications, three panes is a good thing to shoot for, but there are no absolutes here—in fact, many applications require more. On web pages, anything more than two navigation areas and one content area begins to get busy. On tablet apps, two panes is typical.
- Limit the number of controls to as few as your users really need to meet their goals. Having a good grasp of your users via personas will enable you to avoid functions and controls that your users don't really want or need and that, therefore, only get in their way.
- Minimize scrolling when possible. This means giving supporting panes enough room to display information so that they don't require constant scrolling. Default views of 2D and 3D diagrams and scenes should be such that the user can orient himself without too much panning. Zooming is the most difficult type of navigation for most users (though more straightforward in mobile apps using pinch gestures), so its use should be discretionary, not a requirement.

Many online stores present confusing navigation because the designers have tried to serve everyone with one generic site. If a user buys books but never music from a site, access to the music portion of the site could be deemphasized in the main screen for that user. This makes more room for that user to buy books, and the navigation becomes simpler. Conversely, if he visits his account page frequently, his version of the site should prominently display his account button (or tab).

Provide signposts

In addition to reducing the number of navigable places, another way to enhance users' ability to find their way around is by providing better points of reference—*signposts*. In the same way that sailors navigate by reference to shorelines or stars, users navigate by reference to *persistent objects* placed in a user interface.

Persistent objects, in a desktop world, always include the application's windows. Each application most likely has a main, top-level window. The salient features of that window are also considered persistent objects: menu bars, toolbars, and other palettes or visual features like status bars and rulers. Generally, each window of the interface has a distinctive look that will soon become recognizable.

On the web, similar rules apply. Well-designed websites make careful use of persistent objects that remain constant throughout the shopping experience, especially the top-level navigation bar along the top of the page. Not only do these areas provide clear navigational options, but their consistent presence and layout also help orient customers (see Figure 12-9).

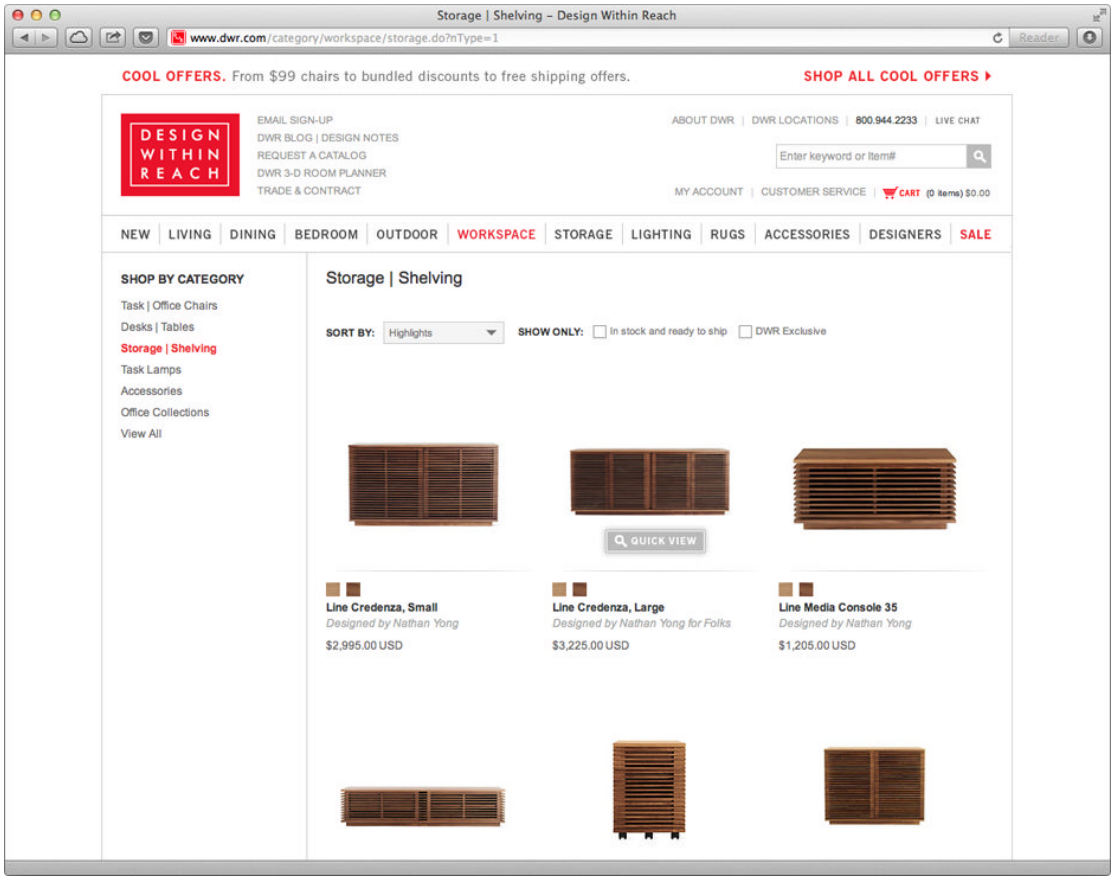


Figure 12-9: The Design Within Reach website makes use of many persistent areas on the majority of its pages, such as the links and search field along the top and the browse tools on the side. These not only help users figure out where they can go but also help keep them oriented.

In devices, similar rules apply to screens, but hardware controls themselves can take on the role of signposts—even more so when they offer visual or tactile feedback about their state. Radio buttons that, for example, light when selected, even a needle's position on a dial, can provide navigational information if integrated appropriately with the software.

Depending on the application, the contents of the application's main window may also be easily recognizable (especially with kiosks and small-screen devices). Some applications may offer a few different views of their data, so the overall aspect of their screens changes depending on the view chosen. A desktop application's distinctive look, however, usually comes from its unique combination of menus, palettes, and toolbars. This means that menus and toolbars must be considered aids to navigation. You don't need a lot of signposts to navigate successfully; they just need to be visible. Needless to say, signposts can't aid navigation if they are removed, so it is best if they are permanent fixtures of the interface (some iOS browsers break this rule slightly by allowing controls to scroll up as the user moves down the page; however, they immediately scroll back down when the user reverses direction—a clever means of bringing controls back into focus as soon as they are needed).

Making each page on a website look just like every other one may maintain visual consistency, but it can, if carried too far, be disorienting. You should use common elements consistently on each page, but by making different rooms look distinctive, you will help orient your users better.

Menus

The most prominent permanent object in a desktop application is the main window and its title and menu bars. Part of the benefit of the menu comes from its reliability and consistency. Unexpected changes to an application's menus can deeply reduce users' trust in them. This is true for menu items as well as for individual menus.

Toolbars

If the application has a toolbar, it should also be considered a recognizable signpost. Because toolbars are idioms for perpetual intermediates rather than for beginners, the strictures against changing menu items don't apply quite as strongly to individual toolbar controls. Removing the toolbar itself is certainly a dislocating change to a persistent object. Although the ability to do so should exist, it shouldn't be offered casually, and users should be protected from accidentally triggering it. Some applications put a control on the toolbar that makes the toolbar disappear! This is a completely inappropriate ejector seat lever.

Other interface signposts

Tool palettes and fixed areas of the screen where data is displayed or edited should also be considered persistent objects that add to the interface's navigational ease. Judicious use of white space and legible fonts is important so that these signposts remain clearly evident and distinctive.

Provide overviews

Overviews serve a purpose similar to signposts in an interface: They help orient users. The difference is that overviews help orient users within the content rather than within the application as a whole. Because of this, the overview area should itself be persistent; its content is dependent on the data being navigated.

Overviews can be graphical or textual, depending on the nature of the content. An excellent example of a graphical overview is the aptly named Navigator palette in Adobe Photoshop, shown in Figure 12-10.

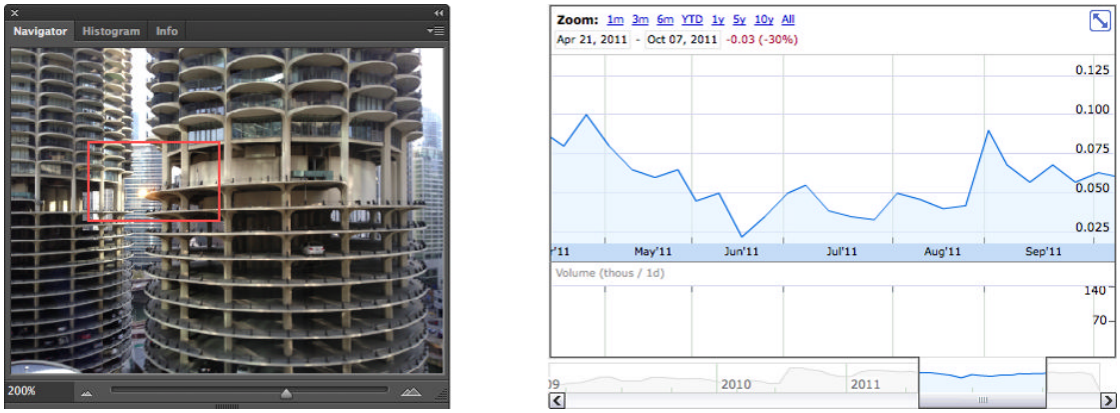


Figure 12-10: On the left, Adobe makes use of an excellent overview idiom in Photoshop: the Navigator palette, which provides a thumbnail view of a large image with an outlined box that represents the portion of the image currently visible in the main display. Not only does the palette provide navigational context, but it can be used to pan and zoom the main display as well. A similar idiom is employed on the right in the Google Finance charting tool, in which the small graph on the bottom provides a big-picture view and context for the zoomed-in view on top.

In the web world, the most common form of overview area is textual: the ubiquitous breadcrumb display (see Figure 12-11). Again, most breadcrumbs provide a navigational aid as well as a navigational control: Not only do they show where in the data structure a visitor is, but they also give him or her tools to move to different nodes in the structure in the form of links. This idiom has lost some popularity as websites have moved from being strictly hierarchical organizations to more associative organizations, which don't lend themselves as neatly to breadcrumbs.

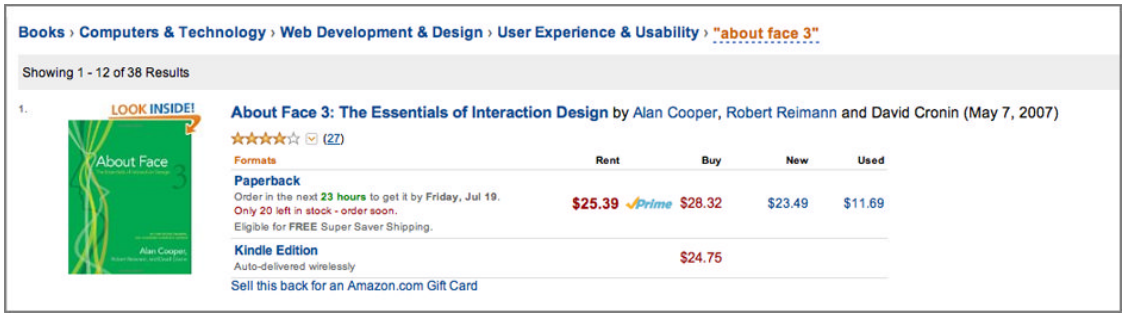


Figure 12-11: A typical breadcrumb display from Amazon.com. Users see where they've been and can click anywhere in the breadcrumb trail to navigate to that link.

A final interesting example of an overview tool is the *annotated scrollbar*, which is most useful for scrolling through text. They make clever use of the linear nature of both scrollbars and textual information to provide location information about the locations of selections, highlights, and potentially many other attributes of formatted or unformatted text. Hints about the locations of these items appear in the "track" that the thumb of the scrollbar moves in, at the appropriate location. When the thumb is over the annotation, the annotated feature of the text is visible in the display. Microsoft Word uses a variation of the annotated scrollbar; it shows the page number and nearest header in a ToolTip that remains active during the scroll, as shown in Figure 12-12.

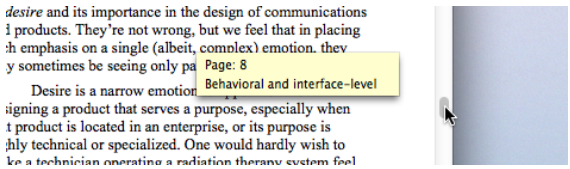


Figure 12-12: An annotated scrollbar from Microsoft Word provides useful context for the user as he or she navigates through a document.

Properly map controls to functions

Mapping describes the relationship between a control, the thing it affects, and the intended result. Poor mapping is evident when a control does not relate visually, spatially, or symbolically to the object it affects. Poor mapping requires users to stop and think about the relationship, breaking flow. Poor mapping of controls to functions increases the cognitive load for users and can result in potentially serious user errors.

Donald Norman provides an excellent example of mapping problems from the non-digital world in *The Design of Everday Things* (Basic Books, 2002). Almost anyone who cooks has run into the annoyance of a stovetop whose burner knobs do not map appropriately to the burners they control. The typical stovetop, such as the one shown in Figure 12-13, features four burners arranged in a flat square with a burner in each corner. However, the knobs that operate those burners are laid out in a straight line on the front of the unit.

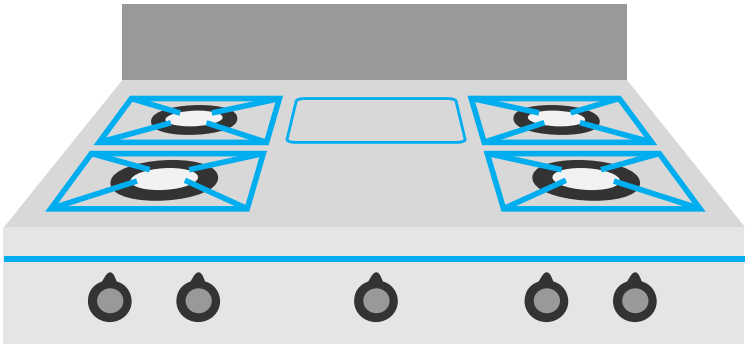


Figure 12-13: A stovetop with poor physical mapping of controls. Does the knob on the far left control the left-front or left-rear burner? Users must figure out the mapping anew each time they use the stovetop.

In this case, we have a *physical mapping* problem. The *result* of using the control is reasonably clear: A burner will heat up when you turn a knob. However, the *target* of the control—which burner will get warm—is unclear. Does twisting the leftmost knob turn on the left-front burner, or does it turn on the left-rear burner? Users must find out by trial and error or by referring to the tiny icons next to the knobs. The unnaturalness of the mapping compels users to figure out this relationship anew every time they use the stove. This cognitive work may become habituated over time, but it still exists, making users prone to error if they are rushed or distracted (as people often are while preparing meals). In the best-case scenario, users feel stupid because they've twisted the wrong knob, and their food doesn't get hot until they notice the error. In the worst-case scenario, they might accidentally burn themselves or set fire to the kitchen.

The solution requires moving the stovetop knobs so that they better suggest which burners they control. The knobs don't have to be laid out in exactly the same pattern as the burners, but they should be positioned so that the target of each knob is clear. The stovetop shown in Figure 12-14 is a good example of an effective mapping of controls.

In this layout, it's clear that the upper-left knob controls the upper-left burner. The placement of each knob visually suggests which burner it will turn on. Norman calls this more intuitive layout "natural mapping."

Figure 12-15 shows another example of poor mapping—of a different type. In this case, it is the *logical mapping* of concepts to actions that is unclear.

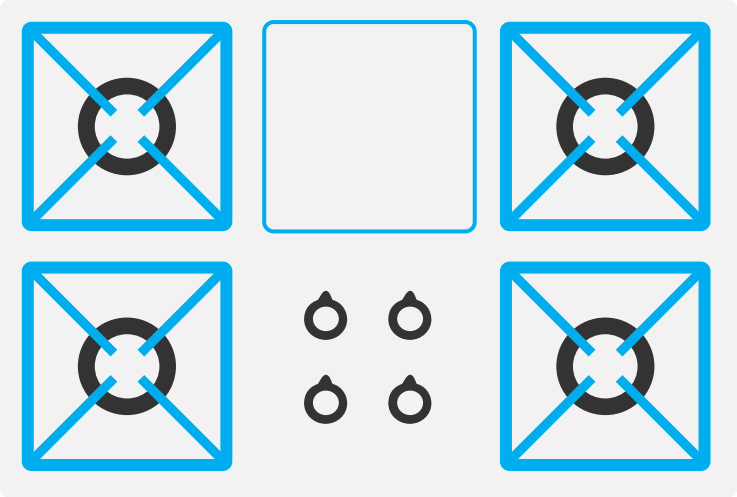


Figure 12-14: Clear spatial mapping. On this stovetop, it is clear which knob maps to which burner, because the spatial arrangement of knobs clearly associates each knob with a burner.

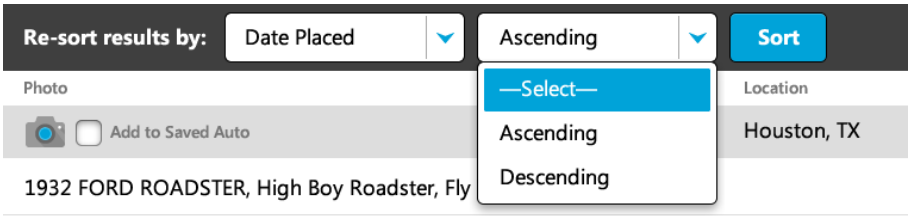


Figure 12-15: An example of a logical mapping problem. If the user wants to see the most recent items first, does she choose Ascending or Descending? These terms don't map well to how users conceive of time.

This website uses a pair of drop-down menus to sort a list of search results by date. The selection in the first drop-down determines the choices present in the second. When **Re-sort results by: Date Placed** is selected in the first menu, the second drop-down presents the options **Ascending** and **Descending**.

Unlike the poorly mapped stovetop knobs, the *target* of this control is clear—the drop-down menu selections affect the list below them. However, the *result* of using the control is unclear: Which sort order will the user get if she chooses Ascending?

The terms chosen to communicate the date-sorting options make it unclear what users should choose if they want to see the most recent items first in the list. **Ascending** and **Descending** do not map well to most users' mental model of time. People don't think of dates as ascending or descending; rather, they think of dates and events as being recent or ancient. A quick fix to this problem is to change the wording of the options to **Most recent first** and **Oldest first**, as shown in Figure 12-16.

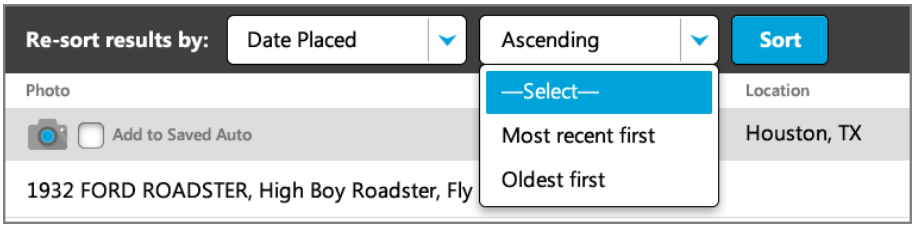


Figure 12-16: Clear, logical mapping. "Most recent" and "Oldest" are terms that users can easily map to time-based sorting.

Whether you make appliances, mobile apps, desktop applications, or websites, your product may have mapping problems. Mapping is an area where attention to detail pays off. You can measurably improve a product by seeking out and fixing mapping problems, even if you have very little time to make changes. The result is a product that is easier to understand and more pleasurable to use.

Avoid hierarchies

Hierarchies are one of the developer's most durable tools. Much of the data inside applications, along with much of the code that manipulates it, is in hierarchical form. For this reason, many developers present hierarchies (the implementation model) in user interfaces. Early menus, as we've seen, were hierarchical. But abstract hierarchies are very difficult for users to successfully navigate, except where they're based on user mental models and the categories are truly mutually exclusive. This truth is often difficult for developers to grasp because they themselves are so comfortable with hierarchies.

Most humans are familiar with hierarchies in their business and family relationships, but hierarchies are not natural concepts for most people when it comes to storing and retrieving arbitrary information. Most mechanical storage systems are simple, composed of either a single sequence of stored objects (like a bookshelf) or a series of sequences, one level deep (like a file cabinet). This method of organizing things into a single layer of groups is extremely common and can be found everywhere in your home and office. Because it never exceeds a single level of nesting, we call this storage paradigm *monocline grouping*.

Developers are comfortable with nested systems, in which an instance of an object is stored in another instance of the same object. Most other humans have a difficult time with this idea. In the mechanical world, complex storage systems, by necessity, use different mechanical form factors at each level. In a file cabinet, you never see folders inside folders or file drawers inside file drawers. Even the dissimilar nesting of folder-inside-drawer-inside-cabinet rarely exceeds two levels of nesting. In the current desktop metaphor used by most window systems, you can nest folder within folder ad infinitum. It's no wonder most computer neophytes get confused when confronted with this paradigm.

Most people store their papers (and other items) in a series of stacks or piles based on some common characteristic: The Acme papers go here; the Project M papers go there; personal stuff goes in the drawer. Donald Norman (1994) calls this a *pile cabinet*. Only inside computers do people put the Project M documents inside the Active Clients folder, which in turn is stored inside the Clients folder, stored inside the Business folder.

Computer science gives us hierarchical structures as tools to solve the very real problems of managing massive quantities of data. But when this implementation model is reflected in the represented model presented to users (as discussed in Chapter 1), they get confused, because it conflicts with their mental model of storage systems. Monocline grouping is the mental model people typically bring to software. Monocline grouping is so dominant outside the computer that interaction designers violate this model at their peril.

Monocline grouping is an inadequate system for physically managing the large quantities of data commonly found on computers, but that doesn't mean it isn't useful as a *represented model*. The solution to this conundrum is to render the structure as the user imagines it—as monocline grouping—but to provide the search and access tools that only a deep hierarchical organization can offer. In other words, rather than forcing users to navigate deep, complex tree structures, give them tools to *bring appropriate information to themselves*. We'll discuss some design solutions that help make this happen in Chapter 14.

Don't replicate Mechanical-Age models

As already discussed, skeuomorphic excise—resulting from an unreflective replication of Mechanical-Age actions in digital interfaces—adds excise, navigational and otherwise.

It makes sense to spend some time rethinking products and features that are translated from the pre-digital world. How can the new, digital version be streamlined and adapted to take full advantage of the digital environment? How can excise be eliminated and smarts brought to bear?

Take the desk calendar. In the non-digital world, calendars are made of paper and are usually divided into a one-month-per-page format. This is a reasonable compromise based on the size of paper, file folders, briefcases, and desk drawers.

Digital products with representations of calendars are quite common, and they almost always display one month at a time. Even if they can show more than one month, as Outlook does, they almost always display days in discrete one-month chunks. Why?

Paper calendars show a single month because they are limited by the size of the paper, and a month is a convenient breaking point. High-resolution digital displays are not so constrained, but most designers copy the mechanical artifact faithfully, as shown in Figure 12-17.

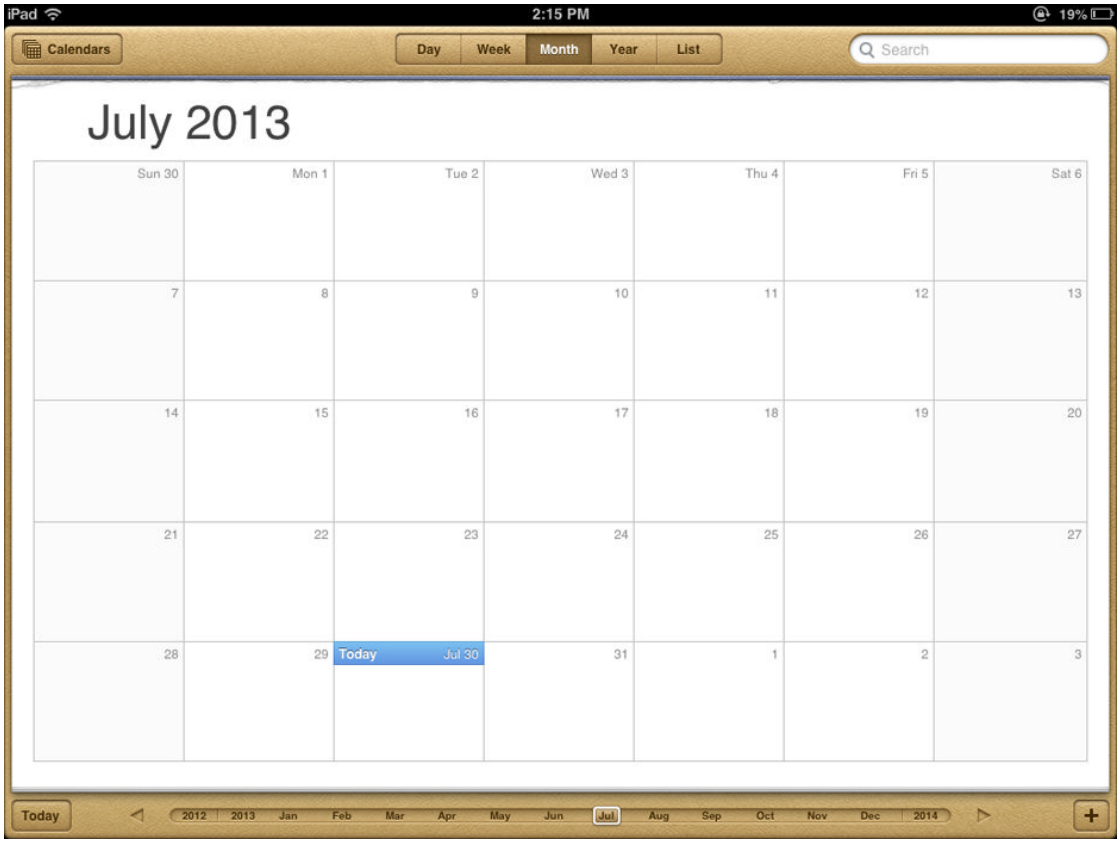


Figure 12-17: The ubiquitous calendar is so familiar that we rarely stop to apply Information-Age sensibilities to its design on the screen. Calendars were originally designed to fit on stacked sheets of paper, not interactive digital displays. How would you redesign a digital calendar? Which of its aspects are artifacts of its old, Mechanical-Age platform?

On an interactive screen, the calendar could easily be a continuously scrolling sequence of days, weeks, or months, as shown in Figure 12-18. Scheduling something from August 28 to September 4 would be simple if weeks were contiguous instead of broken up by the arbitrary monthly division.

Similarly, the grid pattern in digital calendars is almost universally a fixed size. Why can't the width of columns of days or the height of rows of weeks be adjustable like a spreadsheet? Certainly you'd want to adjust the sizes of your weekends to reflect their relative importance in relation to your weekdays. If you're a businessperson, your working-week calendar would demand more space than a vacation week. The adjustable grid interface idiom is well known—every spreadsheet in the world uses it—but the mechanical representations of calendars are so firmly entrenched that we rarely see apps that deviate from them.

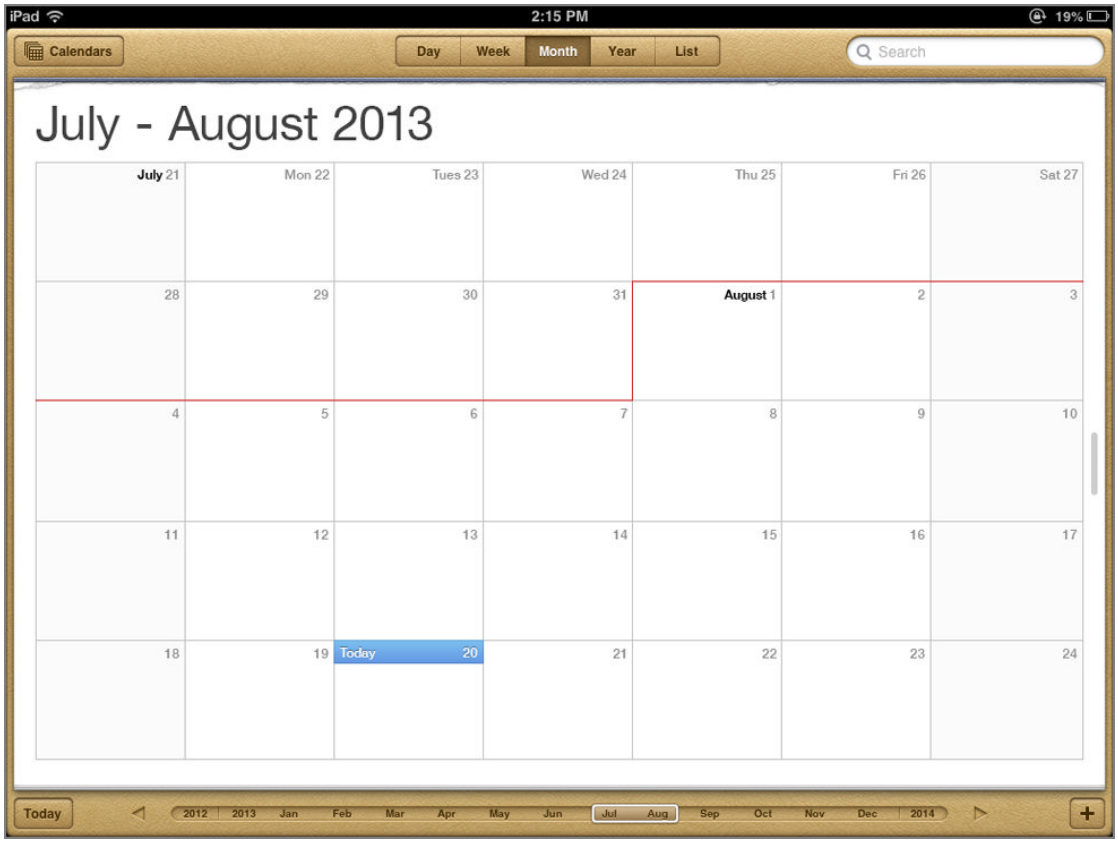


Figure 12-18: Scrolling is a familiar task to computer users. Why not replace the page-oriented calendar with a scrolling one to make it better? This perpetual calendar can do everything the old one can, and it also solves the mechanical-representation problem of scheduling across monthly boundaries. Don't drag old limitations onto new platforms out of habit. What other improvements can you think of?

The designer of the software shown in Figure 12-17 probably thought of calendars as canonical objects that couldn't be altered from the familiar. Surprisingly, most time-management software handles time internally—in its implementation model—as a continuum, and renders it as discrete months only in its user interface—its represented model!

Some might argue that the one-month-per-page calendar is better because it is easily recognizable and familiar to users. However, the new digital model isn't all that different from the old paper model, except that it permits the users to do something they couldn't easily do before—schedule across monthly boundaries. People don't find it difficult to adapt to new representations *if they offer a significant improvement.*

Apple flubbed an opportunity to take this approach with their redesigned iOS7 Calendar app. It features a continuous vertically scrolling calendar in month view... but the designers chose to line break at month boundaries, and didn't support a drag gesture to specify a multi-day event. So close, and yet so far.

DESIGN PRINCIPLE

Significant change must be significantly better.

Paper-style calendars in mobile devices and desktops are mute testimony to how our Mechanical-Age modes of thinking influence our designs. If we don't inform our assumptions about product use with an analysis of user goals, we will end up building excise-ridden software that remains in the Mechanical Age. Better software is based on Information-Age thinking.

Other Common Excise Traps

You should be vigilant in finding and rooting out each small item of excise in your interface. These myriad little extra unnecessary steps can add up to a lot of extra work for users. This list should help you spot excise transgressions:

- Don't force users to go to another window to perform a function that affects the current window.
- Don't force users to remember where they put things in the hierarchical file system.
- Don't force users to resize windows unnecessarily. When a child window pops up on the screen, the application should size it appropriately for its contents. Don't make it big and empty or so small that it requires constant scrolling.
- Don't force users to move windows. If there is open space on the desktop, put the application there instead of directly over some other already open application.
- Don't force users to reenter their personal settings. If the user has ever set a font, color, indentation, or sound, make sure that she doesn't have to do so again unless she wants a change.
- Don't force users to fill in fields to satisfy some arbitrary measure of completeness. If the user wants to omit some details from the transaction entry screen, don't force him to enter them. Assume that he has a good reason for not doing so. In most instances, the completeness of the database isn't worth badgering users over.
- Don't force users to ask permission. This is frequently a symptom of not allowing input in the same place as output.
- Don't ask users to confirm their actions. This requires a robust Undo facility.
- Don't let the user's actions result in an error.

Excise represents the most common and most pernicious barriers to usability and user satisfaction in digital products. Don't let it rear its ugly head in your designs or applications!